SQL CASESTUDY

Here we are analysing the data of employee tables and their related tables and finding the data required for us.

Table structure

-created database
create database salary
use salary

-created table Departments
create table Departments(
dep_id INT PRIMARY KEY,
dept_name VARCHAR(50) NOT NULL UNIQUE
);

-created table Locations
create table Locations(
location_id INT PRIMARY KEY,
location_name VARCHAR(50) NOT NULL UNIQUE
);

-created table Job_roles
create table Job_Roles(
role_id INT PRIMARY KEY,
role_name VARCHAR(50) NOT NULL,
min_salary DECIMAL(10,2),
Max_salary DECIMAL(10,2)
);

-created table Employees
create table Employees(
emp_id INT PRIMARY KEY,
emp_name VARCHAR(100) NOT NULL,
gender VARCHAR(100),
dept_id INT,
role_id INT,
location_id INT,
join_date DATE,
status VARCHAR(20) DEFAULT 'Active',
FOREIGN KEY(dept_id) REFERENCES Departments(dep_id),
FOREIGN KEY(role_id) REFERENCES Job_Roles(role_id),
FOREIGN KEY(location_id) REFERENCES Locations(location_id)
);

```
drop table locations
create table Locations(
location_id INT PRIMARY KEY,
location_name VARCHAR(50) NOT NULL
);

-created table salaries
create table Salaries(
salary_id INT PRIMARY KEY,
emp_id INT,
salary DECIMAL(10,2),
effective_date DATE,
FOREIGN KEY(emp_id) REFERENCES Employees(emp_id)
);

-created table Performance_Reviews
create table Performance_Reviews(
review_id INT PRIMARY KEY,
emp_id INT,
review_year INT,
rating DECIMAL(2,1) CHECK (rating BETWEEN 1 AND 5),
FOREIGN KEY (emp_id) REFERENCES Employees(emp_id)
);


Insert data
INSERT INTO Departments VALUES
(1,'HR'),
(2,'Finance'),
(3,'IT'),
(4,'Marketing'),
(5,'Operations'),
(6,'Sales'),
(7,'Admin'),
(8,'R&D');
INSERT INTO Locations VALUES
(1,'Bangalore'),
(2,'Mumbai'),
(3,'Delhi'),
(4,'Hyderabad'),
(5,'Chennai'),
(6,'Pune');

INSERT INTO Job_Roles VALUES
(1,'HR Executive',25000,50000),
(2,'Financial Analyst',30000,70000),
(3,'Software Engineer',40000,120000),
```

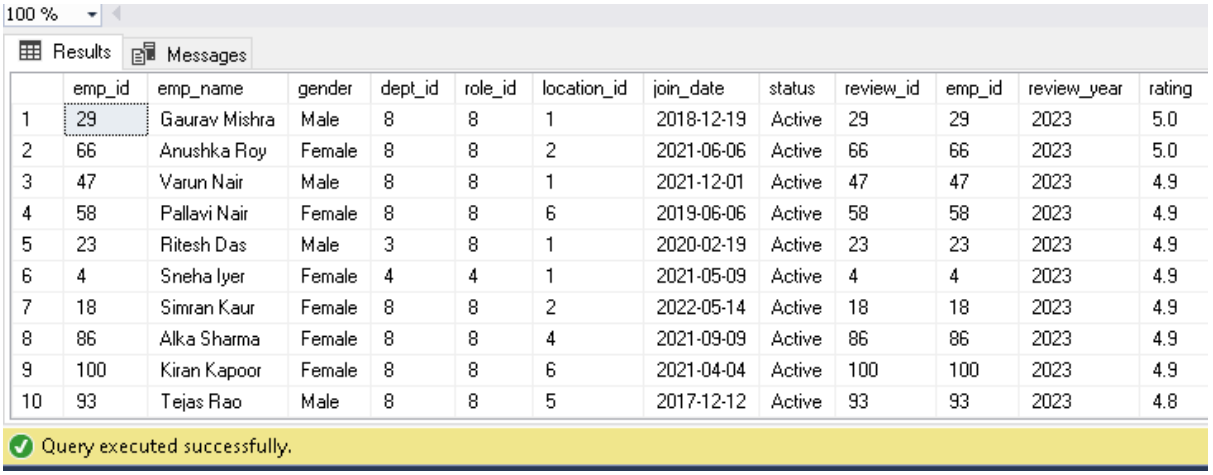
```
(4,'Marketing Manager',35000,90000),
(5,'Operations Manager',40000,100000),
(6,'Sales Executive',20000,60000),
(7,'System Admin',35000,80000),
(8,'Data Scientist',50000,150000);

INSERT INTO Employees VALUES
(1,'Aarav Sharma','Male',3,3,1,'2020-01-15','Active'),
(2,'Diya Verma','Female',1,1,2,'2019-03-20','Active'),
(3,'Rohan Mehta','Male',2,2,3,'2018-07-11','Active'),
(4,'Sneha Iyer','Female',4,4,1,'2021-05-09','Active'),
(5,'Arjun Rao','Male',3,8,4,'2022-02-14','Active'),
(6,'Priya Nair','Female',6,6,5,'2020-06-18','Active'),
(7,'Vikram Singh','Male',5,5,6,'2017-11-25','Active'),
(8,'Ananya Das','Female',3,3,1,'2023-01-01','Active'),
(9,'Karan Malhotra','Male',7,7,2,'2016-09-13','Inactive'),
(10,'Meera Joshi','Female',8,8,3,'2019-12-19','Active')


BULK INSERT Employees
FROM 'C:/Users/Administrator/downloads/employeee.csv'
WITH
(
   FIELDTERMINATOR = ',',
   ROWTERMINATOR = '\n',
   FIRSTROW = 2
);
```

ANALYSIS

1. Top 10 high-performing employees

Ordering data based on rating

```
SELECT TOP 10 *
FROM Employees E
JOIN Performance_Reviews PR ON E.emp_id = PR.emp_id
ORDER BY PR.rating DESC;
```



100 %

Results Messages

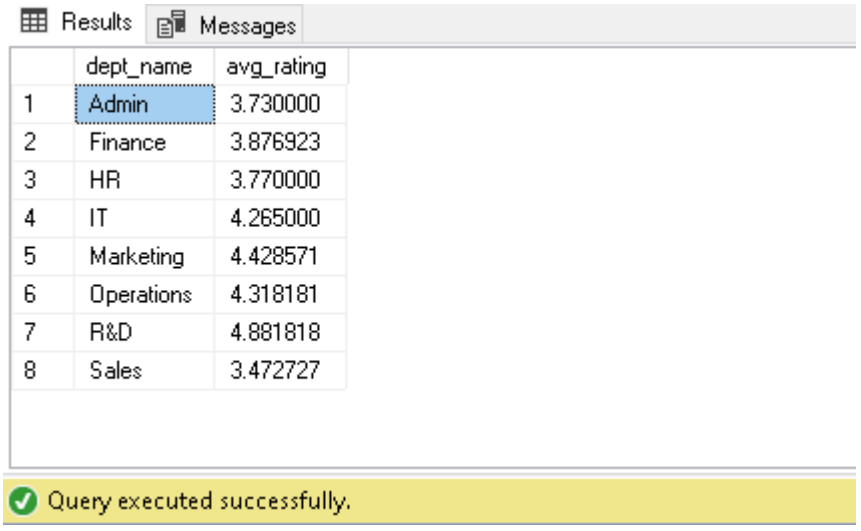
	emp_id	emp_name	gender	dept_id	role_id	location_id	join_date	status	review_id	emp_id	review_year	rating
1	29	Gaurav Mishra	Male	8	8	1	2018-12-19	Active	29	29	2023	5.0
2	66	Anushka Roy	Female	8	8	2	2021-06-06	Active	66	66	2023	5.0
3	47	Varun Nair	Male	8	8	1	2021-12-01	Active	47	47	2023	4.9
4	58	Pallavi Nair	Female	8	8	6	2019-06-06	Active	58	58	2023	4.9
5	23	Ritesh Das	Male	3	8	1	2020-02-19	Active	23	23	2023	4.9
6	4	Sneha Iyer	Female	4	4	1	2021-05-09	Active	4	4	2023	4.9
7	18	Simran Kaur	Female	8	8	2	2022-05-14	Active	18	18	2023	4.9
8	86	Alka Sharma	Female	8	8	4	2021-09-09	Active	86	86	2023	4.9
9	100	Kiran Kapoor	Female	8	8	6	2021-04-04	Active	100	100	2023	4.9
10	93	Tejas Rao	Male	8	8	5	2017-12-12	Active	93	93	2023	4.8

Query executed successfully.

2. Average rating per department

The query calculates the average performance rating for each department by joining employee, review, and department tables.

```
SELECT D.dept_name, AVG(PR.rating) AS avg_rating
FROM Employees E
JOIN Performance_Reviews PR ON E.emp_id = PR.emp_id
JOIN Departments D ON E.dept_id = D.dep_id
GROUP BY D.dept_name;
```



Results Messages

	dept_name	avg_rating
1	Admin	3.730000
2	Finance	3.876923
3	HR	3.770000
4	IT	4.265000
5	Marketing	4.428571
6	Operations	4.318181
7	R&D	4.881818
8	Sales	3.472727

Query executed successfully.

3. Employees eligible for promotion (rating > 4 & 2+ years)

The following query selects the details of employees who have a performance rating greater than 4 and have been employed for 2 or more years.

```
select * from Employees e
join Performance_Reviews
PR ON e.emp_id = PR.emp_id
where pr.rating>4 AND
```

DATEDIFF(YEAR, e.join_date, GETDATE()) >= 2;

00 %

Results | Messages

	emp_id	emp_name	gender	dept_id	role_id	location_id	join_date	status	review_id	emp_id	review_year	rating
1	1	Aarav Sharma	Male	3	3	1	2020-01-15	Active	1	1	2023	4.5
2	3	Rohan Mehta	Male	2	2	3	2018-07-11	Active	3	3	2023	4.2
3	4	Sneha Iyer	Female	4	4	1	2021-05-09	Active	4	4	2023	4.9
4	5	Arjun Rao	Male	3	8	4	2022-02-14	Active	5	5	2023	4.7
5	8	Ananya Das	Female	3	3	1	2023-01-01	Active	8	8	2023	4.1
6	10	Meera Joshi	Female	8	8	3	2019-12-19	Active	10	10	2023	4.8
7	11	Rahul Kapoor	Male	3	3	1	2019-02-10	Active	11	11	2023	4.2
8	14	Pooja Reddy	Female	4	4	4	2021-08-20	Active	14	14	2023	4.6
9	15	Manish Gupta	Male	5	5	5	2017-12-01	Active	15	15	2023	4.3
10	18	Simran Kaur	Female	8	8	2	2022-05-14	Active	18	18	2023	4.9
11	19	Nikhil Verma	Male	3	3	3	2020-10-25	Active	19	19	2023	4.1
12	20	Isha Patel	Female	4	4	4	2021-01-30	Active	20	20	2023	4.5
13	23	Ritesh Das	Male	3	8	1	2020-02-19	Active	23	23	2023	4.9
14	25	Harsh Vardh...	Male	5	5	3	2016-08-17	Active	25	25	2023	4.2
15	26	Tanvi Joshi	Female	4	4	4	2022-09-10	Active	26	26	2023	4.4
16	27	Amit Yadav	Male	3	3	5	2019-03-14	Active	27	27	2023	4.1

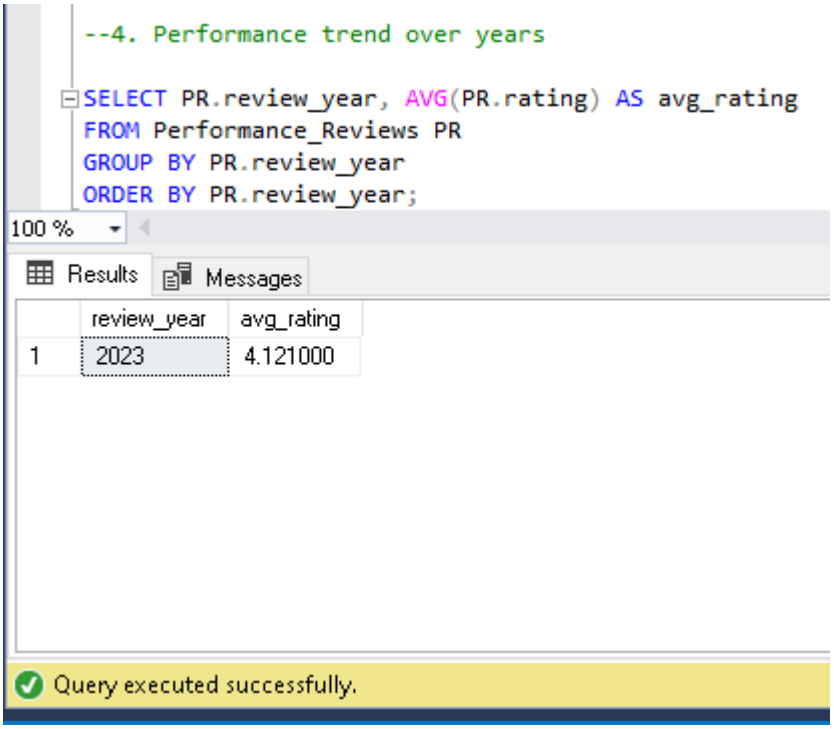
Query executed successfully.

4. Performance trend over years

4\. Performance trend over years

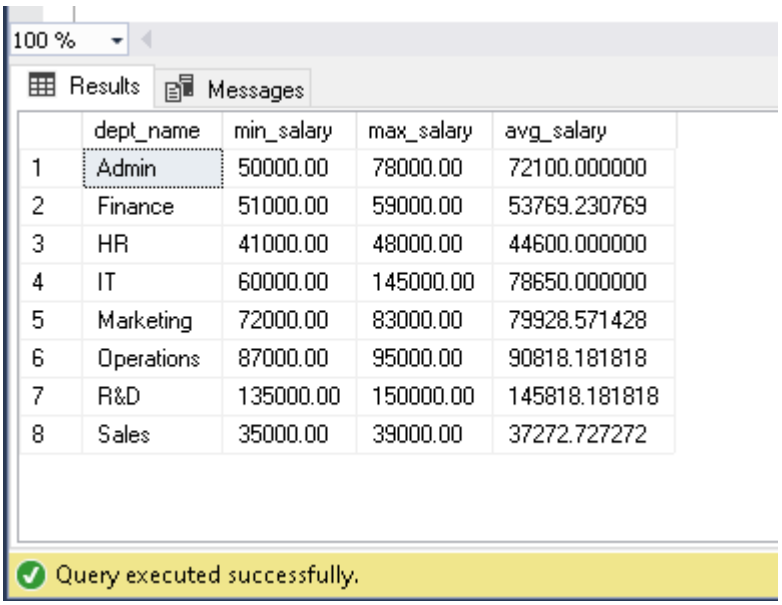
The query calculates the average performance rating across all employees for each review year to show the general trend of performance over time.

```
SELECT PR.review_year, AVG(PR.rating) AS avg_rating
FROM Performance_Reviews PR
GROUP BY PR.review_year
ORDER BY PR.review_year;
SELECT PR.review_year, AVG(PR.rating) AS avg_rating
FROM Performance_Reviews PR
GROUP BY PR.review_year
ORDER BY PR.review_year;
```



5. Salary Distribution per Department

```
select d.dept_name,
MIN(S.salary) AS min_salary,
MAX(S.salary) AS max_salary,
AVG(S.salary) AS avg_salary
from Employees e
join Salaries S
on s.emp_id=e.emp_id
join Departments d
on d.dep_id=e.dept_id
group by d.dept_name
```



100 %

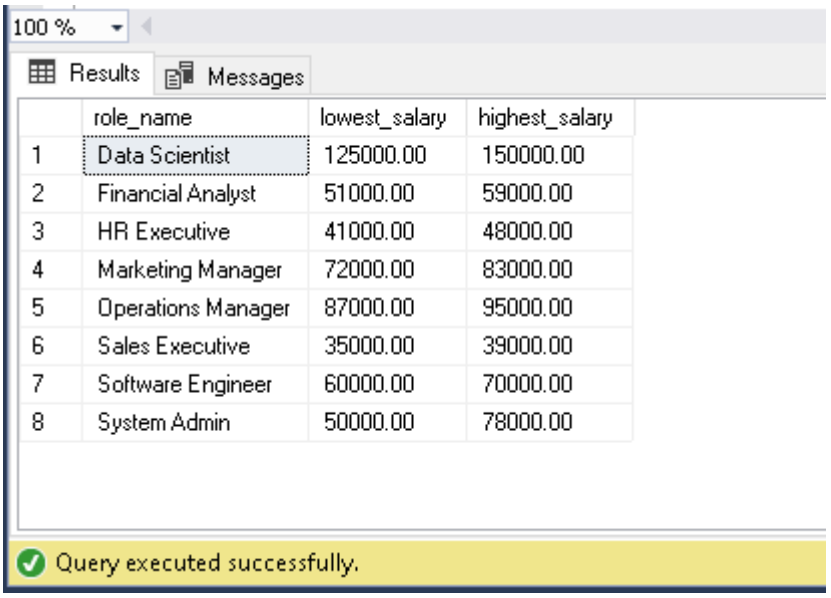
Results | Messages

	dept_name	min_salary	max_salary	avg_salary
1	Admin	50000.00	78000.00	72100.000000
2	Finance	51000.00	59000.00	53769.230769
3	HR	41000.00	48000.00	44600.000000
4	IT	60000.00	145000.00	78650.000000
5	Marketing	72000.00	83000.00	79928.571428
6	Operations	87000.00	95000.00	90818.181818
7	R&D	135000.00	150000.00	145818.181818
8	Sales	35000.00	39000.00	37272.727272

Query executed successfully.

6. Highest & lowest salary per role

```
select r.role_name,
MIN(S.salary) AS lowest_salary,
MAX(S.salary) AS highest_salary
from Employees e
join Salaries S
on s.emp_id=e.emp_id
join Job_Roles r
on r.role_id=e.role_id
group by r.role_name
```



100 %

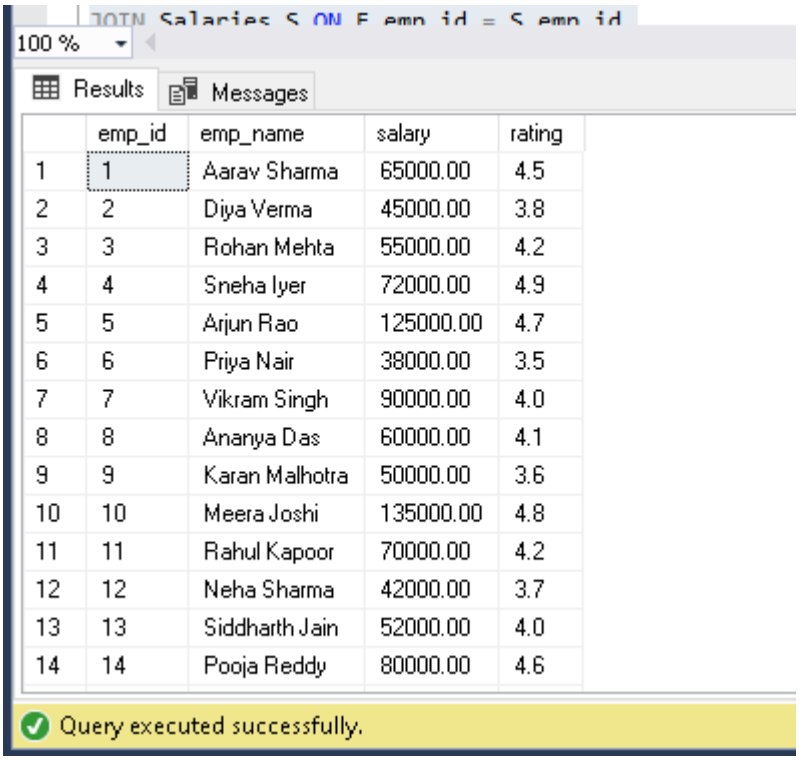
Results | Messages

	role_name	lowest_salary	highest_salary
1	Data Scientist	125000.00	150000.00
2	Financial Analyst	51000.00	59000.00
3	HR Executive	41000.00	48000.00
4	Marketing Manager	72000.00	83000.00
5	Operations Manager	87000.00	95000.00
6	Sales Executive	35000.00	39000.00
7	Software Engineer	60000.00	70000.00
8	System Admin	50000.00	78000.00

Query executed successfully.

7. Salary vs Performance Correlation

```
SELECT E.emp_id, E.emp_name, S.salary, PR.rating
FROM Employees E
JOIN Salaries S ON E.emp_id = S.emp_id
JOIN Performance_Reviews PR ON E.emp_id = PR.emp_id;
```



100 %

Results | Messages

	emp_id	emp_name	salary	rating
1	1	Aarav Sharma	65000.00	4.5
2	2	Diya Verma	45000.00	3.8
3	3	Rohan Mehta	55000.00	4.2
4	4	Sneha Iyer	72000.00	4.9
5	5	Arjun Rao	125000.00	4.7
6	6	Priya Nair	38000.00	3.5
7	7	Vikram Singh	90000.00	4.0
8	8	Ananya Das	60000.00	4.1
9	9	Karan Malhotra	50000.00	3.6
10	10	Meera Joshi	135000.00	4.8
11	11	Rahul Kapoor	70000.00	4.2
12	12	Neha Sharma	42000.00	3.7
13	13	Siddharth Jain	52000.00	4.0
14	14	Pooja Reddy	80000.00	4.6

Query executed successfully.

8. Employees Below Department Average Salary

```
WITH DeptAvgSalary AS (
    SELECT E.dept_id, AVG(S.salary) AS avg_salary
    FROM Employees E
    JOIN Salaries S ON E.emp_id = S.emp_id
    GROUP BY E.dept_id
)
SELECT E.emp_id, E.emp_name, S.salary, D.dept_name
FROM Employees E
JOIN Salaries S ON E.emp_id = S.emp_id
JOIN Departments D ON E.dept_id = D.dep_id
JOIN DeptAvgSalary DAS ON E.dept_id = DAS.dept_id
WHERE S.salary < DAS.avg_salary;
```

Results Messages

	emp_id	emp_name	salary	dept_name
1	1	Aarav Sharma	65000.00	IT
2	4	Sneha Iyer	72000.00	Marketing
3	7	Vikram Singh	90000.00	Operations
4	8	Ananya Das	60000.00	IT
5	9	Karan Malhotra	50000.00	Admin
6	10	Meera Joshi	135000.00	R&D
7	11	Rahul Kapoor	70000.00	IT
8	12	Neha Sharma	42000.00	HR
9	13	Siddharth Jain	52000.00	Finance
10	16	Kavya Nair	36000.00	Sales
11	18	Simran Kaur	140000.00	R&D
12	19	Nikhil Verma	61000.00	IT
13	21	Deepak Singh	53000.00	Finance
14	22	Anjali Mehta	41000.00	HR

Query executed successfully.

9. Hiring Trend per Year

```
SELECT YEAR(E.join_date) AS hire_year, COUNT(E.emp_id) AS headcount
FROM Employees E
GROUP BY YEAR(E.join_date)
ORDER BY hire_year;
```

100 %

Results Messages

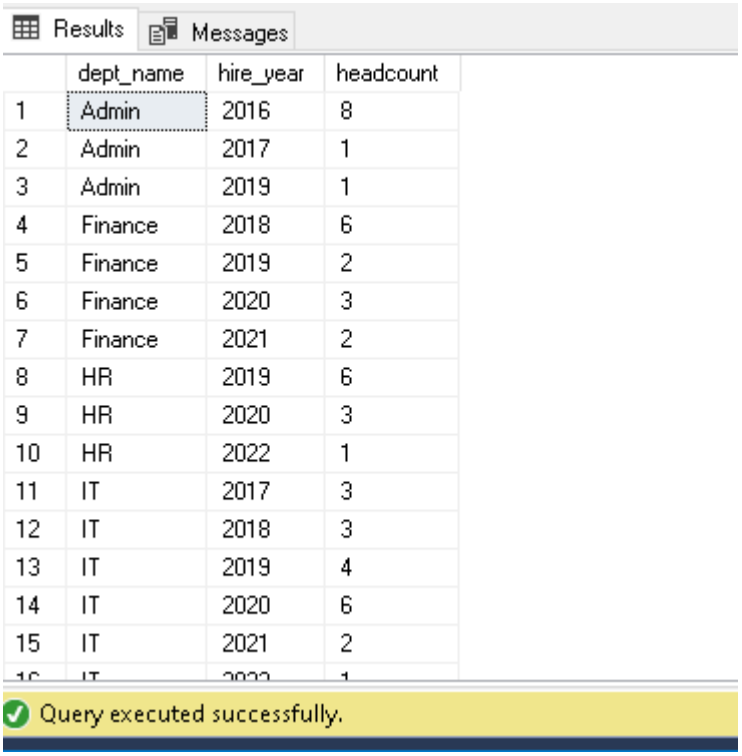
	hire_year	headcount
1	2016	10
2	2017	14
3	2018	14
4	2019	20
5	2020	16
6	2021	15
7	2022	10
8	2023	1

Query executed successfully.

10.Headcount growth by department

```
SELECT D.dept_name,
YEAR(E.join_date) AS hire_year,
COUNT(E.emp_id) AS headcount
FROM Employees E
```

```
JOIN Departments D ON E.dept_id = D.dep_id
GROUP BY D.dept_name, YEAR(E.join_date)
ORDER BY D.dept_name, hire_year;
```



Results | Messages

	dept_name	hire_year	headcount
1	Admin	2016	8
2	Admin	2017	1
3	Admin	2019	1
4	Finance	2018	6
5	Finance	2019	2
6	Finance	2020	3
7	Finance	2021	2
8	HR	2019	6
9	HR	2020	3
10	HR	2022	1
11	IT	2017	3
12	IT	2018	3
13	IT	2019	4
14	IT	2020	6
15	IT	2021	2

Query executed successfully.

12. Employees overdue for promotion

```
select * from Employees e
join Performance_Reviews
PR ON e.emp_id = PR.emp_id
where pr.rating>4 AND
DATEDIFF(YEAR , e.join_date, GETDATE())>=2;
```

Results | Messages

	emp_id	emp_name	gender	dept_id	role_id	location_id	join_date	status	review_id	emp_id	review_year	rating
1	1	Aarav Sharma	Male	3	3	1	2020-01-15	Active	1	1	2023	4.5
2	3	Rohan Mehta	Male	2	2	3	2018-07-11	Active	3	3	2023	4.2
3	4	Sneha Iyer	Female	4	4	1	2021-05-09	Active	4	4	2023	4.9
4	5	Arjun Rao	Male	3	8	4	2022-02-14	Active	5	5	2023	4.7
5	8	Ananya Das	Female	3	3	1	2023-01-01	Active	8	8	2023	4.1
6	10	Meera Joshi	Female	8	8	3	2019-12-19	Active	10	10	2023	4.8
7	11	Rahul Kapoor	Male	3	3	1	2019-02-10	Active	11	11	2023	4.2
8	14	Pooja Reddy	Female	4	4	4	2021-08-20	Active	14	14	2023	4.6
9	15	Manish Gupta	Male	5	5	5	2017-12-01	Active	15	15	2023	4.3
10	18	Simran Kaur	Female	8	8	2	2022-05-14	Active	18	18	2023	4.9
11	19	Nikhil Verma	Male	3	3	3	2020-10-25	Active	19	19	2023	4.1
12	20	Isha Patel	Female	4	4	4	2021-01-30	Active	20	20	2023	4.5
13	23	Ritesh Das	Male	3	8	1	2020-02-19	Active	23	23	2023	4.9
14	25	Harsh Vardh...	Male	5	5	3	2016-08-17	Active	25	25	2023	4.2
15	26	Tanvi Joshi	Female	4	4	4	2022-09-10	Active	26	26	2023	4.4
16	27	Amit Yadav	Male	3	3	5	2019-03-14	Active	27	27	2023	4.1
17	29	Gaurav Mis...	Male	8	8	1	2018-12-19	Active	29	29	2023	5.0

Query executed successfully.

13.High salary but low performance cases

```
SELECT E.emp_id, E.emp_name, S.salary, PR.rating
FROM Employees E
JOIN Salaries S ON E.emp_id = S.emp_id
JOIN Performance_Reviews PR ON E.emp_id = PR.emp_id
WHERE S.salary > (select avg(salary) from Salaries)
AND PR.rating < 4;
```

Results | Messages

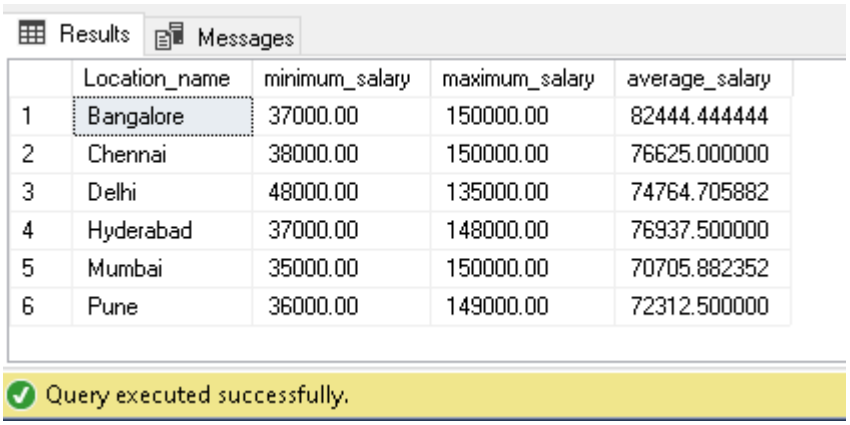
	emp_id	emp_name	salary	rating
1	17	Aditya Roy	78000.00	3.9
2	37	Sanjay Kulkarni	76000.00	3.6
3	67	Ravindra Singh	83000.00	3.5
4	77	Puneet Sharma	76000.00	3.6
5	99	Omkar Das	76000.00	3.8

Query executed successfully.

16 location-wise salary comparison

```
select l.Location_name,min(salary) as minimum_salary,max(salary) as
maximum_salary,avg(salary) as average_salary from Employees e
join Locations l
```

```
on e.location_id=l.location_id
join Salaries s
on e.emp_id=s.emp_id
group by l.location_name
```



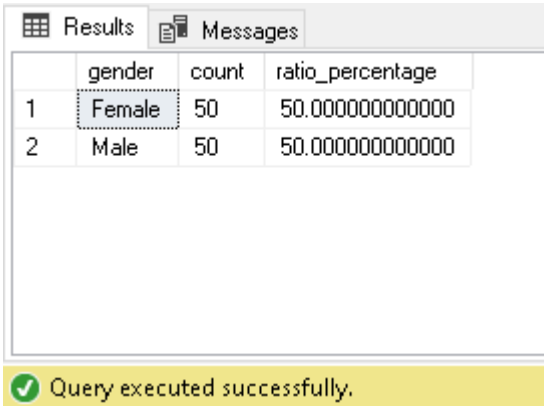
Results | Messages

	Location_name	minimum_salary	maximum_salary	average_salary
1	Bangalore	37000.00	150000.00	82444.444444
2	Chennai	38000.00	150000.00	76625.000000
3	Delhi	48000.00	135000.00	74764.705882
4	Hyderabad	37000.00	148000.00	76937.500000
5	Mumbai	35000.00	150000.00	70705.882352
6	Pune	36000.00	149000.00	72312.500000

Query executed successfully.

17. gender diversity ratio

```
SELECT gender,COUNT(*) AS count,
ROUND( (COUNT(*) * 100.0) / (SELECT COUNT(*) FROM Employees), 2) AS
ratio_percentage
FROM Employees
GROUP BY gender;
```



Results | Messages

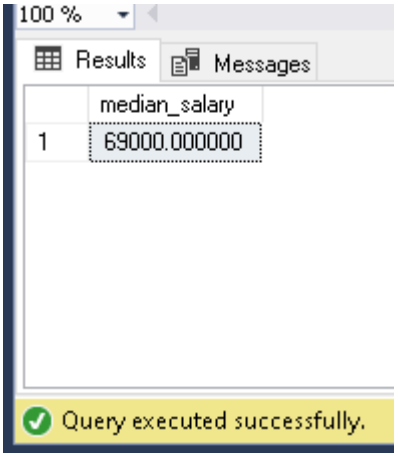
	gender	count	ratio_percentage
1	Female	50	50.000000000000
2	Male	50	50.000000000000

Query executed successfully.

18 median salary calculation

```
WITH SalaryRank AS (
SELECT salary, ROW_NUMBER() OVER (ORDER BY salary) AS RowAsc,
ROW_NUMBER() OVER (ORDER BY salary DESC) AS RowDesc
FROM Salaries
)
SELECT AVG(salary) AS median_salary
FROM SalaryRank
WHERE RowAsc = RowDesc
```

OR RowAsc + 1 = RowDesc;



20 identify leadership pipeline candidate

20. Identify leadership pipeline candidate

The query identifies high-performing employees (rating >= 4.5) who are currently in managerial, lead, or senior roles, suggesting potential candidates for leadership development.

```
SELECT E.emp_id, E.emp_name, PR.rating, JR.role_name
FROM Employees E
JOIN Performance_Reviews PR ON E.emp_id = PR.emp_id
JOIN Job_Roles JR ON E.role_id = JR.role_id
WHERE PR.rating >= 4.5  -- High performers
AND (JR.role_name LIKE '%Manager%' OR JR.role_name LIKE '%Lead%' OR
JR.role_name LIKE '%Senior%');
```